PersistHotStuff: A BFT Consensus with Persistence, Dynamic Membership, and Pluggable State Machines

1. PROBLEM STATEMENT

1.1 Background

Distributed systems deployed in adversarial environments (e.g., blockchains, permissioned ledgers, consortium databases) require Byzantine Fault Tolerant (BFT) consensus protocols. HotStuff, introduced by Yin et al. (2019), is a modern leader-based BFT consensus protocol designed with two key properties:

- **Linearity**: O(n) communication per decision, unlike earlier quadratic protocols such as PBFT. For N+1 replicas, the leader sends N proposal messages and receives up to N votes, resulting in linear communication complexity = 2N.
- **Responsiveness**: After the network becomes synchronous, progress is limited only by actual network delay, not by conservative timeout estimates.

The protocol uses a **3-chain commit rule**: a block is considered committed when it becomes the great-grandparent in a chain of three consecutive blocks, each justified by a quorum certificate (QC; signatures from at least 2/3 of replicas). This ensures safety under Byzantine faults.

1.2 Identified Limitations

The Stanford CS244B “HotStuff Implementation and Advice” report (Spring 2024) implements HotStuff as a teaching and evaluation prototype, and explicitly highlights four critical limitations:

Limitation L1: No Persistence
The implementation operates entirely in memory; if a replica process crashes, its state is lost. This affects system safety when multiple crashes occur because replicas may diverge in their view of committed blocks.

Impact: Complete loss of all state (blocks, QCs, executed commands) on node failure. No fault tolerance across reboots.

Limitation L2: Static Membership
The system assumes a fixed set of replicas (n = 3f + 1). There is no mechanism to add or remove replicas while the system is running, limiting scalability and adaptability.

Impact: Cannot perform live scaling, cluster reconfiguration is manual and error-prone, limited fault tolerance recovery.

Limitation L3: Simple State Machine
The implementation uses a simple log-based state machine. There is no generic mechanism to plug in richer application logic such as key–value stores, counters, or blockchain semantics.

Impact: Single-purpose prototype, difficult to extend, code reuse across applications is minimal.

Limitation L4: Pacemaker/Liveness Issues
When there are few or no client requests, progress may stop because the chain cannot grow to the length required by the 3-chain commit rule.

Impact: System freezes when no external input arrives, single-client workloads cause indefinite stalls.

1.3 Problem Definition

Given:
- A BFT consensus protocol following the HotStuff design
- A set of n = 3f + 1 replicas, up to f of which may be Byzantine
- A partially synchronous network

Challenge: How to design and implement an extended HotStuff framework that addresses all four limitations simultaneously while maintaining Byzantine safety and liveness properties?

---

2. OBJECTIVES

2.1 Primary Objectives

O1 - Implement Crash-Safe Persistence
Design and implement Write-Ahead Logging (WAL) and periodic snapshots enabling replicas to recover consistent state after crashes, with recovery time bounded by O(log_size + snapshot_size).

O2 - Enable Dynamic Membership
Develop join and leave protocols using quorum certificates, allowing the set of replicas to change over time (4-7 replicas) while maintaining the invariant n' = 3f' + 1 and preserving Byzantine safety through quorum intersection.

O3 - Create Pluggable State Machine Interface
Implement a trait-based application interface allowing the consensus core to execute commands against arbitrary application state machines (KV store, counter, blockchain logic) without modifying the core consensus algorithm.

O4 - Improve Liveness Under Low Load
Enhance the pacemaker to introduce dummy proposals when necessary, ensuring the system makes progress even with a single client or temporarily no client requests, without breaking safety.

2.2 Secondary Objectives

- Document the design and implementation for reproducibility and future research
- Release code as open-source under MIT license for community adoption
- Provide comprehensive testing covering crash recovery, membership changes, and liveness scenarios

---

3. EXPECTED OUTCOMES

3.1 Functional Outcomes

- The system continues to **operate correctly after replica crashes**, with every node able to recover its last committed state from the write-ahead log and snapshots without losing any committed commands.

- The cluster supports **online addition and removal of replicas** through membership-change commands agreed by consensus, while maintaining n = 3f + 1 and quorum intersection conditions required for Byzantine safety.

- The consensus core can be **reused across applications** by implementing the application trait, allowing different state machines (KV store, counter, token-transfer logic) without modifying the HotStuff core code.

- Under **low or single-client load**, the protocol no longer stalls: dummy proposals allow the 3-chain rule to be satisfied, so even a trickle of commands is eventually committed.

3.2 Quantitative Outcomes

Crash Recovery: - Tolerate **up to 1 crash out of 4 replicas** (n = 4, f = 1) while preserving a single, consistent committed log on recovery - Each restarted replica should rebuild state and rejoin within **≤ 5 seconds** for a log of **≤ 10,000 commands** with snapshots taken every **1,000 commands** - Run minimum of **100 crash-recovery cycles** with **zero committed-state divergences** across replicas

Dynamic Membership: - Support online reconfiguration between **4 and 7 replicas** (e.g., 4→5→6→7 and back down) - Complete at least **50 membership changes** in tests without any observed safety violation - Maintain the invariant n' = 3f' + 1 across all reconfigurations

---

4. METHODOLOGY

4.1 Overall Approach

We will implement PersistHotStuff as a from-scratch Rust implementation of the HotStuff BFT consensus protocol, using existing projects like `hotstuff_rs` (ParallelChain Lab) only as architectural inspiration. The system will be structured into modular layers:

- **Consensus Core**: Block structures, voting, QC formation, commit rules
- **Storage Layer**: Persistence via WAL and snapshots
- **Membership Manager**: Dynamic validator set changes
- **Application Interface**: Generic trait for pluggable state machines

The development follows an incremental approach with four main stages, each validated independently before integration.

4.2 Stage 1: Core Consensus Protocol

Objective: Implement the basic chained HotStuff protocol with a static set of validators.

Key Components:

- **Block Structure**: Parent references forming a tree/DAG, payload containing commands, view number, proposer signature

- **Quorum Certificates (QC)**: Aggregated signatures from 2f+1 replicas, proving quorum approval
- **Replica State**: Locked QC, highest QC seen, last voted view, block tree
- **3-Chain Commit Rule**: A block commits when it becomes the great-grandparent in a justified chain
- **Leader Rotation**: Round-robin or view-based leader selection
- **Voting Protocol**: Replicas vote for proposals that extend their locked QC

Implementation Approach:
- Use Rust's type system to enforce safety properties (immutable blocks, strong typing)
- Start with in-memory simulation: multiple replicas as async tasks communicating via channels
- Leverage `tokio` async runtime for concurrent replica execution
- Implement message passing with configurable delays and drops for testing

Validation:
- Unit tests for voting rules, QC formation, and commit detection
- Simulation tests with 4 replicas under normal operation
- Verify all honest replicas commit the same sequence

4.3 Stage 2: Persistence and Crash Recovery

Objective: Add durable storage so replicas can recover state after crashes.

Key Components:

- **Write-Ahead Log (WAL)**: Append-only log recording blocks, QCs, and votes with sequence numbers and checksums
- **Snapshots**: Periodic checkpoints of committed state (block tree + application state)
- **Recovery Protocol**: On restart, load latest snapshot and replay subsequent WAL entries

Implementation Approach:
- Design WAL entry format: sequence number, entry type, serialized payload, CRC32 checksum
- Use file-based storage with `fsync()` after each append for durability
- Snapshot every N committed commands (e.g., N=1000)
- Truncate WAL after successful snapshot to bound storage growth
- Use `serde` and `bincode` for efficient serialization

Validation:
- Kill replicas at random points during operation
- Restart and verify recovered state matches expected committed log
- Run 100+ crash-recovery cycles with zero divergence
- Test recovery with various log sizes and snapshot intervals

---

4.4 Stage 3: Dynamic Membership

Objective: Allow validator set to change over time through consensus-driven reconfiguration.

Key Components:

- **Validator Set**: Data structure mapping public keys to voting power, computing quorum thresholds
- **Membership Commands**: `AddValidator`, `RemoveValidator`, `UpdateStake` as special consensus commands
- **Quorum Intersection**: Ensure old and new validator sets overlap sufficiently to prevent forks
- **State Catch-Up**: New validators sync committed history before participating

Implementation Approach:
- Treat membership changes as normal consensus commands proposed by leader
- Apply configuration updates only after the enclosing block is committed (3-chain rule)
- Maintain invariant: total voting power satisfies $n \geq 3f + 1$
- New validators request blocks from existing replicas and download snapshots
- Update quorum threshold dynamically based on current validator set

Validation:
- Test online addition and removal during active client workload
- Verify safety across 50+ membership changes (4→5→6→7→6→5→4)
- Test boundary cases: minimum validators (n=4), maximum (n=7)
- Ensure no safety violations during configuration transitions

---

4.5 Stage 4: Application Interface and Liveness

Objective: Decouple consensus from application logic and ensure progress under low load.

Application Interface:

- **Trait Definition**: Generic `App` trait with methods:
  - `apply(command) → result`: Execute command on state machine
  - `snapshot() → state`: Capture current application state
  - `restore(state)`: Restore from snapshot

- **Concrete Implementations**:
  - Key-Value Store: PUT/GET/DELETE operations
  - Counter (optional): INCREMENT/GET operations

- **Separation of Concerns**: Consensus orders commands; application defines semantics

Liveness Improvements:

- **Pacemaker Enhancement**: Track view timeouts and uncommitted block count

- **Dummy Proposals**: When timeout expires with < 3 uncommitted blocks and no client commands, leader proposes no-op dummy commands

- **Dummy Handling**: Consensus layer treats dummies as normal blocks; application layer filters them (no state change)

Implementation Approach:
- Define trait with clear semantics and error handling
- Implement KV store with in-memory HashMap

- Enhance pacemaker to monitor progress and inject dummies when needed
- Ensure dummy commands don't violate safety (still go through 3-chain commit)

Validation:
- Single-client scenarios: issue one command, verify it commits
- Idle periods: no commands for 1 minute, system remains stable
- Verify dummy proposals trigger correctly and enable commits
- Test application swapping without consensus code changes

4.6 Testing and Validation Strategy

Unit Testing:
- Test individual components in isolation (voting, QC formation, storage operations)
- Use Rust's `cargo test` framework
- Achieve high code coverage for critical paths

Integration Testing:
- Multi-replica simulations with 4-7 replicas
- Inject faults: message delays, drops, crashes, membership changes
- Run combined scenarios: crashes during membership changes, low-load conditions
- Validate end-to-end correctness

Property-Based Testing (optional):
- Use `proptest` or `quickcheck` to generate random message sequences
- Verify invariants: safety (no conflicting commits), eventual progress

Stress Testing:
- Long-running tests (10+ minutes) with continuous workload
- Measure qualitative performance: commit latency, recovery time
- Document observations for future optimization

4.7 Technology Stack

- **Programming Language**: Rust (stable, via `rustup`)
- **Async Runtime**: `tokio` (concurrency and future networking)
- **Serialization**: `serde` + `bincode` (efficient binary encoding)
- **Cryptography**: `ed25519-dalek` (digital signatures), `sha2` (hashing)
- **Logging**: `tracing` + `tracing-subscriber` (structured logging)

- **Testing**: `cargo test`, `proptest` (optional property-based tests)
- **Version Control**: Git + GitHub (public repository)
- **Documentation**: `cargo doc`, README, architecture docs

Design Inspiration (conceptual reference only):
- **hotstuff_rs** (ParallelChain Lab): Architecture and module organization
- **Raft**: WAL and snapshot patterns for crash recovery
- **PBFT**: Dummy message approach for liveness
- **Tendermint**: Validator set dynamics and application separation

Note: All code is written from scratch; no existing codebase is forked or extended.

---

5. TIMELINE (16 WEEKS)

Weeks	Goals	Activities
1-2	Problem understanding & design	Study HotStuff and BFT, review literature, finalize architecture and roadmap
3-4	Core data structures & voting logic	Implement replicas, blocks, votes, QCs, and quorum-based voting logic
5-6	Proposals & commit rule	Implement leader proposals, block validation, and 3-chain commit logic
7	Liveness & view changes	Implement pacemaker, leader rotation, and timeout-based view changes

8-9	Visualization & simulation	Build block-tree visualization and multi-replica simulations
10-11	Crash-safe persistence	Add write-ahead logging, recovery, and snapshot support
12	Dynamic membership	Implement QC-based join and leave operations
13	Improved liveness	Add dummy proposals for progress under low load
14	Messaging layer	Introduce message passing between replicas
15	Evaluation and analysis	Measure performance and compare with baseline HotStuff
16	Finalization	Documentation, final report, demo